

Thuật toán khớp nhiều xâu và những gợi ý từ nó

Zhu Zeyuan
Trường Ngoại ngữ Nam Kinh

Nguồn gốc: Multiple string matching algorithms and their implications

IOI China candidate team papers / 2004 / Zhu Zeyuan.pdf

Từ khóa

Xâu mẫu; cây tiền tố từ; cây hậu tố; khớp xâu.

Trích yếu

Xử lý xâu có vị trí quan trọng trong ứng dụng thực tế. Nó trông có vẻ đơn giản, nhưng khi nghiên cứu sâu hơn, nhiều tư tưởng tinh tế xuất hiện trong đó và độ khó cũng trở nên rất lớn. Vì vậy, các kỳ thi tin học thường dùng bài toán xử lý xâu để rèn luyện năng lực sáng tạo của thí sinh.

Bài viết nêu bài toán và phân tích trực tiếp ban đầu ở Chương 1. Chương 2, 3 và 5 lần lượt giới thiệu ba công cụ chuẩn bị: thuật toán khớp mẫu KMP, cấu trúc cây tiền tố từ do tác giả xây dựng, và thuật toán cây hậu tố. Dựa trên tư tưởng cốt lõi của KMP, Chương 4 đưa ra một thuật toán tuyến tính cho bài toán khớp nhiều xâu. Bài viết không dừng ở độ phức tạp tuyến tính; Chương 6 tiếp tục đề xuất một thuật toán có hiệu năng trung bình tốt hơn. Cuối cùng, Chương 7 phân tích cách hình thành thuật toán và nâng tư tưởng ấy lên mức phương pháp luận.

Contents

1 Đặt vấn đề	1
1.1 Mô tả bài toán	1
1.2 Những ý tưởng ban đầu	2
2 Thuật toán Knuth–Morris–Pratt	2
2.1 Định nghĩa	2
2.2 Hàm tiền tố của mẫu	3
2.3 Thuật toán chính của KMP	3
3 Cây tiền tố từ	3
3.1 Định nghĩa trie	3
3.2 Xây dựng trie	4
3.3 Định nghĩa con trở tiền tố	4

3.4	Sinh con trở tiền tố	4
4	Thuật toán chính thứ nhất: thuật toán tuyến tính	5
4.1	Gợi ý từ KMP	5
4.2	Dùng cây tiền tố từ và nhãn phụ	5
4.3	Độ phức tạp của cây tiền tố từ	5
4.4	Quy trình chính	6
4.5	Độ phức tạp thời gian và bộ nhớ	6
4.6	Một số mở rộng	7
5	Cây hậu tố và thuật toán McCreight	7
5.1	Cấu trúc dữ liệu	7
5.2	Một số định nghĩa	7
5.3	Xây cây hậu tố: khung ban đầu	8
5.4	Liên kết hậu tố	8
5.5	Xây cây hậu tố	9
6	Thuật toán chính thứ hai: thuật toán có hiệu năng trung bình tốt hơn	9
6.1	Dùng và mở rộng cây tiền tố từ	9
6.2	Dùng và mở rộng cây hậu tố	10
6.3	Hai hàm trên TreeA và TreeB	10
6.4	Quy trình chính	10
6.5	Một ví dụ	11
6.6	Phân tích độ phức tạp	11
7	Gợi ý và kết luận	13
7.1	Phân tích thuật toán	13
7.2	Gợi ý phương pháp	14
7.3	Kết luận	14

1 Đặt vấn đề

1.1 Mô tả bài toán

Khớp nhiều xâu nghĩa là: cho một số xâu mẫu, trong một văn bản chỉ gồm 26 chữ cái thường a đến z, tìm vị trí xuất hiện đầu tiên của bất kỳ xâu mẫu nào. Cụ thể, cho m xâu mẫu

$$P_1[1..L_1], P_2[1..L_2], \dots, P_m[1..L_m]$$

và văn bản $T[1..n]$. Với ràng buộc

$$\sum L \leq 100K, \quad m \leq 1000, \quad n \leq 900K,$$

ta cần tìm số nguyên nhỏ nhất $s \in [1, n]$ sao cho tồn tại $a \in [1, m]$ thỏa

$$\forall x \in [1, L_a], \quad T[s + x - 1] = P_a[x].$$

Nếu các mẫu là cdefg và efg, còn văn bản là abcdefgh, yêu cầu “tìm tất cả vị trí xuất hiện của tất cả mẫu” dễ làm mục tiêu trở nên không rõ ràng khi chỉ cần một câu trả lời. Vì vậy ta thường chuyển sang tìm “vị trí đầu tiên”. Các thuật toán bên dưới vẫn có thể liệt kê mọi lần xuất hiện mà không đổi bậc độ phức tạp.

Một ứng dụng thực tế là công cụ tìm kiếm có từ khóa logic. Trong sinh học, khi tìm một vài mẫu có thể là biến dị trong chuỗi DNA, ta cũng gặp bài toán cùng loại. Do đó, giải bài toán này bằng thuật toán hiệu quả có thể cải thiện đáng kể hiệu suất trong nhiều lĩnh vực.

1.2 Những ý tưởng ban đầu

Cách đơn giản nhất là xét từng vị trí rồi thử từng mẫu:

```
for i <- 1 to n do
  for j <- 1 to m do
    if i + L[j] - 1 <= n then
      if T[i..i+L[j]-1] = P[j][1..L[j]] then
        output i and stop
```

Thuật toán này kiểm tra các vị trí theo thứ tự tăng dần. Trong trường hợp xấu nhất, độ phức tạp thời gian là

$$O\left(n \sum L\right).$$

Một cải tiến tự nhiên hơn là tìm lần xuất hiện đầu tiên của từng mẫu trong văn bản:

```
for i <- 1 to m do
  X[i] <- first occurrence of P[i] in T, or infinity if absent
output min(X)
```

Vị trí xuất hiện đầu tiên của P_i trong T có thể tìm bằng KMP trong $O(n + L_i)$, do đó tổng độ phức tạp là

$$O\left(nm + \sum L\right).$$

Hai phương pháp này vẫn quá yếu trước quy mô dữ liệu đã nêu. Ta cần cố gắng tìm một thuật toán tuyến tính, hoặc còn tốt hơn. Trước hết ta giới thiệu KMP, thuật toán khớp một xâu, và một cấu trúc dữ liệu chuẩn bị là cây tiền tố từ.

2 Thuật toán Knuth–Morris–Pratt

Thuật toán này được D. E. Knuth, J. H. Morris và V. R. Pratt độc lập phát hiện, thường gọi tắt là KMP.

2.1 Định nghĩa

Cho mẫu $P[1..m]$ và văn bản $T[1..n]$. Ta cần tìm mọi số nguyên $s \in [1, n - m + 1]$ sao cho

$$\forall x \in [1, m], \quad T[s + x - 1] = P[x].$$

2.2 Hàm tiền tố của mẫu

Với $1 \leq i \leq m$, định nghĩa hàm tiền tố

$$\pi(i) = \max\{j \mid 0 \leq j < i, P[1..j] = P[i-j+1..i]\}.$$

Nói cách khác, $\pi(i)$ là độ dài biên dài nhất của $P[1..i]$, tức một tiền tố đồng thời là hậu tố thật sự của đoạn này.

Hàm này có thể được tiền xử lý bằng đoạn sau:

```
k <- 0
pi[1] <- 0
for i <- 2 to m do
  while k > 0 and P[k+1] != P[i] do
    k <- pi[k]
  if P[k+1] = P[i] then
    k <- k + 1
  pi[i] <- k
```

Vì k tăng nhiều nhất $m - 1$ lần, số lần thực hiện phép gán $k \leftarrow pi[k]$ cũng không vượt quá $m - 1$. Vì vậy đoạn tiền xử lý có độ phức tạp $O(m)$.

2.3 Thuật toán chính của KMP

Hàm tiền tố ghi lại thông tin tự khớp sau khi dịch của chính mẫu. Khi gặp ký tự không khớp, ta có thể dựa vào hàm tiền tố để điều chỉnh vị trí mẫu thay vì so sánh lại từ đầu.

Ví dụ, nếu đang khớp được một đoạn dài 5 và $\pi(5) = 3$, mẫu có thể dịch sang phải $5 - 3 = 2$ ô rồi tiếp tục so sánh. Nếu vẫn không khớp, ta lại tiếp tục dùng π để dịch.

Tương tự cách tính mảng tiền tố, thuật toán khớp chính có dạng:

```
q <- 0
for i <- 1 to n do
  while q > 0 and P[q+1] != T[i] do
    q <- pi[q]
  if P[q+1] = T[i] then
    q <- q + 1
  if q = m then
    output an occurrence ending at i
  q <- pi[q]
```

Giống phần tính hàm tiền tố, số lần thực hiện $q \leftarrow pi[q]$ không vượt quá n . Vì vậy toàn bộ KMP chạy trong

$$O(n + m).$$

3 Cây tiền tố từ

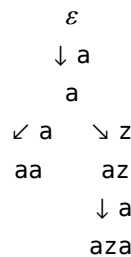
3.1 Định nghĩa trie

Cây tiền tố từ thực chất là một trie. Trong trường hợp lý tưởng, đó là một cây 26 phân vô hạn; 26 cạnh đi xuống từ mỗi nút được gán nhãn lần lượt là a, b, ..., z.

Với mỗi nút p , ghép các chữ cái trên đường đi từ gốc đến p ta được một xâu, ký hiệu S_p . Ngược lại, với một xâu S , ta đi theo các chữ cái của S trong trie và đến một nút cuối, ký hiệu p_S . Như vậy trong trie đầy đủ có tương ứng một-một giữa nút và xâu.

3.2 Xây dựng trie

Trong thực tế ta không thể xây một trie vô hạn. Ta bắt đầu từ cây rỗng và lần lượt chèn những từ cần dùng. Chẳng hạn, sau khi chèn aa rồi aza, phần hữu hạn của trie chứa các đường:



Quy trình chèn một xâu $S[1..n]$ là:

```

p <- Root
for i <- 1 to n do
  if p.Child[S[i]] = NIL then
    create and initialize p.Child[S[i]]
  p <- p.Child[S[i]]

```

Ở đây $p.Child[ch]$, với ch thuộc $a..z$, là con của p theo cạnh mang nhãn ch .

3.3 Định nghĩa con trở tiền tố

Cây tiền tố từ khác trie thường ở chỗ mỗi nút không phải gốc có thêm một con trở tiền tố. Với nút p , tìm số nguyên nhỏ nhất

$$k \in [2, |S_p|]$$

sao cho $S_p[k..|S_p|]$ tồn tại trong cây. Khi đó con trở tiền tố của p trở đến nút

$$p_{S_p[k..|S_p|]}.$$

Ví dụ, xét nút ứng với abab. Ta thử hậu tố bab; nếu bab không xuất hiện trong trie, ta thử tiếp ab. Nếu ab có trong trie, con trở tiền tố của abab trở đến nút ab. Các nút rỗng không được biểu diễn trong các sơ đồ này; con trở tiền tố nên hiểu như một cạnh phụ không thuộc cây.

3.4 Sinh con trở tiền tố

Nếu tính con trở tiền tố bằng cách dò lại từ định nghĩa, độ phức tạp sẽ rất lớn. Bắt chức hàm tiền tố trong KMP, ta có thể dùng con trở tiền tố của nút cha để suy ra con trở tiền tố của nút hiện tại. Gọi nút được con trở tiền tố của p trở tới là nút tiền tố của p .

Giả sử đang xử lý nút p và mọi nút có độ sâu nhỏ hơn đã có con trở tiền tố. Ta tính con trở của p như sau:

```

q <- p.Father
ch <- label of the edge q -> p
q <- q.Prefix
while q != Root and q.Child[ch] = NIL do
  q <- q.Prefix
if q.Child[ch] = NIL then
  p.Prefix <- Root
else
  p.Prefix <- q.Child[ch]

```

Nói bằng lời: từ nút tiền tố của cha, thử đi xuống bằng đúng chữ cái cuối cùng dẫn vào p ; nếu không có cạnh đó, tiếp tục đi theo các con trở tiền tố cho đến khi tìm được hoặc về gốc.

4 Thuật toán chính thứ nhất: thuật toán tuyến tính

4.1 Gợi ý từ KMP

Tinh túy của KMP là giảm tính toán trùng lặp: dựa vào sự tự khớp sau khi dịch của mẫu để xác định cần dịch mẫu bao nhiêu. Với bài toán nhiều mẫu, ta cũng muốn chỉ quét văn bản một lần. Khi gặp ký tự không khớp, ta cần tìm hậu tố dài nhất có thể của tiền tố hiện tại. Hậu tố ấy không nhất thiết phải là tiền tố của cùng một mẫu; nó có thể là tiền tố của bất kỳ mẫu nào.

Ví dụ, văn bản có đoạn đầu zababaa...; mẫu ababaca thất bại sau khi đã khớp ababa. Ta không nhất thiết dịch mẫu này sang phải 2 ô. Nếu có mẫu khác như babaab, ta có thể dịch ít hơn và tiếp tục từ tiền tố của mẫu mới. Dịch càng ít càng tốt mới là điều cần thiết.

Các mẫu liên hệ với nhau khá chặt chẽ. Để hạ độ phức tạp cài đặt, cấu trúc cây là một lựa chọn tự nhiên. Kết hợp với tư tưởng hàm tiền tố của KMP, ta thu được cây tiền tố từ đã nêu ở Chương 3.

4.2 Dùng cây tiền tố từ và nhãn phụ

Ta xây cây tiền tố từ TreeA bằng cách chèn toàn bộ mẫu, rồi tính các con trở tiền tố. Khi nào thì ta thật sự tìm thấy một mẫu P_a ? Cách đơn giản nhất là đặt nhãn phụ Okay tại nút p_{P_a} , với giá trị a , để ghi nhận rằng đây là điểm kết thúc của mẫu a ; các nút khác có Okay = 0.

Chỉ đánh dấu như vậy vẫn chưa đủ. Nếu có hai mẫu abcd và bc, khi đang ở nút abcd, mẫu bc cũng vừa kết thúc, vì bc là một hậu tố có mặt trong trie. Nút bc chính là nút tiền tố của abcd. Vì vậy khi sinh con trở tiền tố, ta truyền nhãn Okay: nếu nút tiền tố của nút hiện tại có nhãn Okay, nút hiện tại cũng phải ghi nhận nhãn ấy. Trong cài đặt để tìm mọi mẫu, một nút nên lưu danh sách các mẫu kết thúc tại nó thay vì chỉ một giá trị.

4.3 Độ phức tạp của cây tiền tố từ

Trong quá trình sinh con trở tiền tố, ta dùng đúng tư tưởng như khi sinh mảng tiền tố KMP. Có thể chứng minh độ phức tạp bằng phân tích thế năng.

Với một mẫu $P[1..n]$, đặt p_i là nút tương ứng với $P[1..i]$ trong trie, và p_0 là gốc. Ta chứng minh tổng chi phí tính con trở tiền tố cho $p_0, p_1, \dots, p_n$ là $O(n)$.

Gọi c_i là chi phí tính con trở tiền tố của p_i , với $c_0 = 1$. Định nghĩa thế năng $\Phi(i)$ của p_i như sau:

$$\Phi(0) = 0, \quad \Phi(i) = \text{độ sâu của nút tiền tố của } p_i \quad (1 \leq i \leq n).$$

Rõ ràng p_i là cha của p_{i+1} khi $0 \leq i < n$. Trong giả mã sinh con trở, mỗi lần vòng lặp đi theo $q.\text{Prefix}$, độ sâu của q giảm ít nhất 1. Do đó số lần lặp khi tính p_{i+1} nhiều nhất là

$$\Phi(i) + 1 - \Phi(i + 1).$$

Suy ra

$$c_{i+1} \leq \Phi(i) + 2 - \Phi(i + 1).$$

Cộng theo mọi $0 \leq i < n$:

$$\sum_{i=1}^n c_i \leq 2n + \Phi(0) - \Phi(n) \leq 2n.$$

Vì vậy việc tính con trở tiền tố trên n nút này mất $O(n)$ thời gian. Toàn bộ cây tiền tố từ được xây trong

$$O\left(\sum L\right).$$

4.4 Quy trình chính

Bắt chước KMP, thuật toán khớp nhiều xâu đầu tiên là:

```

Algorithm MultiPatternMatching1
  preprocess: build the prefix trie
  q <- Root
  for i <- 1 to n do
    while q != Root and q.Child[T[i]] = NIL do
      q <- q.Prefix
    if q.Child[T[i]] != NIL then
      q <- q.Child[T[i]]
    if q.Okay then
      output every pattern ending at i represented by q.Okay
End

```

Giống KMP, số lần thực hiện $q \leftarrow q.\text{Prefix}$ không vượt quá tổng số lần đi xuống theo cạnh cây. Vì vậy phần quét văn bản chạy trong $O(n)$.

Nếu yêu cầu đúng vị trí bắt đầu nhỏ nhất của một mẫu, cần chú ý rằng mẫu kết thúc sớm hơn chưa chắc bắt đầu sớm hơn. Ví dụ một mẫu dài hơn có thể bắt đầu trước nhưng kết thúc sau một mẫu ngắn. Ta chỉ cần thêm phép so sánh vị trí bắt đầu $i - L_a + 1$ của mọi mẫu vừa được báo, rồi lấy vị trí nhỏ nhất.

4.5 Độ phức tạp thời gian và bộ nhớ

Chương trình chính gồm hai phần: xây cây tiền tố từ và quét văn bản. Tổng thời gian là

$$O\left(\sum L + n\right).$$

Nếu mỗi nút lưu đủ 26 con trở, mỗi nút cần 26 ô con trở. Số nút là $O(\sum L)$, do đó bộ nhớ là

$$O\left(26 \sum L\right).$$

Quét văn bản không cần thêm bộ nhớ đáng kể.

4.6 Một số mở rộng

Nếu văn bản không dài, hoặc bảng chữ cái lớn, bộ nhớ có thể trở thành nút thắt cổ chai. Có hai hướng xử lý.

Thứ nhất, chuyển mọi ký tự thành nhị phân rồi xây cây nhị phân. Ví dụ chữ a có thể được biểu diễn bằng một dãy bit cố định như 01100001. Khi đó cây 26 phân trở thành cây nhị phân. Mức cải thiện không thật lý tưởng vì độ dài biểu diễn tăng lên.

Thứ hai, phân bổ động danh sách con của từng nút và lưu có thứ tự. Khi cần đi theo một ký tự, ta tìm nhị phân trong danh sách con. Thời gian có tăng một nhân tử nhỏ, nhưng bộ nhớ giảm còn

$$O\left(\sum L\right).$$

Nếu yêu cầu mọi vị trí xuất hiện của mọi mẫu, thuật toán vẫn làm được. Điểm khác là tại Okay có thể có nhiều mẫu cùng kết thúc, nên ta lưu một danh sách thay vì một nhãn đơn.

Khi mẫu ngắn nhất rất dài và văn bản gần như ngẫu nhiên, còn có một thuật toán có hiệu năng trung bình tốt hơn. Để trình bày nó, trước hết cần giới thiệu cây hậu tố.

5 Cây hậu tố và thuật toán McCreight

5.1 Cấu trúc dữ liệu

Cây hậu tố cơ bản là trie tạo bởi tất cả hậu tố của một từ. Với từ ababc, ta có năm hậu tố:

$$ababc, \quad babc, \quad abc, \quad bc, \quad c.$$

Nếu xây trie trực tiếp, số nút có thể đạt cấp $O(N^2)$ với N là độ dài từ. Để tránh điều này, ta nén các đường chỉ có một nhánh: mỗi cạnh có thể mang cả một xâu, và các cạnh đi ra từ cùng một nút có ký tự đầu khác nhau. Khi chèn một hậu tố, nhiều nhất chỉ thêm một cạnh mới, nên số nút là $O(N)$.

Một vấn đề nhỏ là có thể có một hậu tố kết thúc ở giữa một cạnh đã nén. Để tránh trường hợp ấy, ta thêm vào cuối xâu một ký tự chưa xuất hiện trong xâu, ví dụ \$. Khi đó không hậu tố nào là tiền tố của hậu tố khác.

5.2 Một số định nghĩa

Cho từ $S[1..n]$. Ký hiệu

$$\text{suf}_i = S[i..n]$$

là hậu tố dài thứ i của S .

Mỗi nút trong cây hậu tố có các trường chính:

- **Father**: con trỏ đến cha;
- **Suffix**: liên kết hậu tố, sẽ định nghĩa bên dưới;
- **Child[c]**: con có cạnh bắt đầu bằng ký tự c ;
- **String[c]**: xâu ghi trên cạnh đến **Child[c]**.

Trong cài đặt, **String[c]** thường được biểu diễn bằng hai chỉ số đầu và cuối trỏ vào xâu gốc S .

5.3 Xây cây hậu tố: khung ban đầu

Thuật toán McCreight xây cây hậu tố T trong n bước. Ban đầu T rỗng; ở bước i , chèn suf_i vào T .

Định nghĩa head_i là tiền tố chung dài nhất, lấy lớn nhất trên mọi suf_j với $1 \leq j < i$, giữa suf_i và suf_j .
Viết

$$\text{suf}_i = \text{head}_i + \text{tail}_i.$$

Khung xây dựng là:

```
for i <- 1 to n do
  find the node corresponding to head[i]
  create p.Child[tail[i][1]] and initialize it
  p.String[tail[i][1]] <- tail[i]
```

Ví dụ với $S = \text{abab\$}$, ta lần lượt chèn $\text{abab\$}$, $\text{bab\$}$, $\text{ab\$}$, $\text{b\$}$, $\text{\$}$. Việc thêm $\text{tail}[i]$ chỉ tốn $O(1)$ nếu đã biết nút của $\text{head}[i]$; nút này mới là phần khó. Nếu mỗi lần đều dò từ gốc, tổng thời gian sẽ rất cao.

5.4 Liên kết hậu tố

Tính chất 1. Nếu head_{i-1} viết được thành $x\delta$, trong đó x là một ký tự và δ có thể rỗng, thì δ chắc chắn là tiền tố của head_i .

Thật vậy, cây rẽ nhánh tại nút ứng với head_{i-1} , nên tồn tại $j < i - 1$ sao cho $x\delta$ là tiền tố của suf_j . Khi đó δ là tiền tố của suf_{j+1} . Đồng thời δ cũng là tiền tố của suf_i , nên nó là tiền tố của head_i .

Liên kết hậu tố được định nghĩa như sau. Với mỗi nút trong không phải lá p , xâu ứng với nó có thể viết là $x\delta$, với x là một ký tự và δ có thể rỗng. Con trỏ $p.\text{Suffix}$ trỏ đến vị trí của δ trong cây; nếu δ rỗng thì trỏ về gốc. Liên kết hậu tố của gốc trỏ đến chính gốc.

Trong quá trình xây, ta duy trì:

- Tính chất 2: ngay sau khi một nút trong không phải lá được tạo, liên kết hậu tố của nó được tính ngay.
- Tính chất 3: khi cần tính liên kết hậu tố của một nút trong, xâu đích của liên kết hậu tố đã xuất hiện trong cây.

Nhờ vậy, tương tự cây tiền tố từ, để xây liên kết hậu tố của nút p ta bắt đầu từ liên kết hậu tố của cha p , rồi đi xuống:

Procedure CountSuffixLink(p)

```
q <- p.Father.Suffix
s <- string on the edge p.Father -> p
if q = Root then
  s <- s[2..|s|]
while s != "" do
  s1 <- q.String[s[1]]
  L <- min(|s|, |s1|)
  if L = |s| and L != |s1| then
    create an internal node r and initialize it
    r.String[s1[L+1]] <- s1[L+1..|s1|]
    r.Child[s1[L+1]] <- q.Child[s[1]]
    q.String[s[1]] <- s
```

```

    q.Child[s[1]] <- r
    p.Suffix <- r
    CountSuffixLink(r)
    return
  q <- q.Child[s[1]]
  s <- s[L+1..|s|]
End

```

Vì tính chất 3 đảm bảo sâu cần tìm đã có trong cây, ở dòng tính L chỉ cần so sánh độ dài phần cạnh liên quan để biết đi tiếp hay tách cạnh.

5.5 Xây cây hậu tố

Khi đã có liên kết hậu tố, mỗi lần tìm head_i chỉ cần bắt đầu từ liên kết hậu tố của nút ứng với head_{i-1} rồi đi xuống. Khung thuật toán trở thành:

```

for i <- 1 to n do
  1. find the node p corresponding to head[i-1]
  2. from p.Suffix, descend to the node q for head[i]
  3. create q.Child[tail[i][1]] and initialize it
  4. q.String[tail[i][1]] <- tail[i]
  5. if p.Suffix != q then CountSuffixLink(q)

```

Phân tích thời gian:

- Các bước 1, 3 và 4 tốn $O(1)$ mỗi vòng, tổng cộng $O(N)$.
- Bước 5 tính liên kết hậu tố cho mọi nút trong. Vì trong `CountSuffixLink` ta dùng độ dài cạnh để đi qua hoặc tách cạnh, tổng chi phí tỉ lệ với số nút, tức $O(N)$.
- Bước 2 tốn $O(|\text{head}_i| - |\text{head}_{i-1}| + 1)$ theo phân tích khấu hao, tổng cộng cũng là $O(N)$.

Vì vậy thuật toán McCreight xây cây hậu tố trong $O(N)$.

6 Thuật toán chính thứ hai: thuật toán có hiệu năng trung bình tốt hơn

6.1 Dùng và mở rộng cây tiền tố từ

Tương tự thuật toán chính thứ nhất, ta dùng cây tiền tố từ `TreeA` xây từ toàn bộ mẫu. Để phù hợp với thuật toán thứ hai, mỗi nút thêm tham số `Shift`. Tham số này ghi độ dài đường đi ngắn nhất từ nút hiện tại đến một nút `Okay` bất kỳ, không kể chính nó. Đường đi có thể dùng cạnh cây hoặc con trở tiền tố; cạnh cây có trọng số 1, con trở tiền tố có trọng số 0.

Ý nghĩa của `Shift`: nếu con trở hiện đang ở nút này, ta cần đọc thêm ít nhất bao nhiêu ký tự nữa mới có thể tạo ra một lần khớp mẫu.

Với các mẫu `abab`, `ba`, `bb`, các nút kết thúc mẫu được đánh dấu `Okay`. Giá trị `Shift` được xây như sau. Khi chèn ký tự thứ j của mẫu P_i , đặt

$$\text{new} = \begin{cases} L_i - j, & 1 \leq j < L_i, \\ L_i, & j = L_i. \end{cases}$$

Nếu tạo nút mới p , gán $p.\text{Shift} \leftarrow \text{new}$. Nếu đi đến một nút đã tồn tại tại p , gán

$$p.\text{Shift} \leftarrow \min(p.\text{Shift}, \text{new}).$$

Như vậy mới xét các cạnh cây, chưa xét con trở tiền tố. Vì con trở tiền tố luôn đi từ tầng sâu lên tầng nông hơn, ta duyệt các nút theo độ sâu tăng dần và cập nhật

$$p.\text{Shift} \leftarrow \min(p.\text{Shift}, p.\text{Prefix}.\text{Shift}).$$

Tổng số nút là $O(\sum L)$, nên xây và duy trì TreeA mất $O(\sum L)$ thời gian.

6.2 Dừng và mở rộng cây hậu tố

Thuật toán thứ hai còn dùng một cây hậu tố TreeB, được xây từ tất cả hậu tố của các mẫu sau khi đảo ngược.

Nếu các mẫu là abab, ba, bb, các xâu đảo lần lượt là baba, ab, bb. Cây hậu tố của chúng cho phép ta kiểm tra từ phải sang trái một đoạn văn bản và biết trong $O(N)$ thời gian liệu đoạn độ dài N ấy có phải là xâu con của một mẫu nào không.

Khác phần trình bày cây hậu tố ở trên, trong mô tả TreeB ta không cần hiện các cạnh chứa ký tự kết thúc $\$$; ký tự ấy chỉ phục vụ quá trình xây dựng.

6.3 Hai hàm trên TreeA và TreeB

```
Function ScanA(L, R, point)
  scan T[L..R] in TreeA, starting from node point
  set point to the last reached node
  while point.Shift < shortest_pattern_length div 2 do
    read one more character from the text
    continue moving point in TreeA
  output every Okay encountered during the process
  return the last text position scanned and the final point
End
```

Quét trong TreeA chính là bước đi của thuật toán ở Chương 4: đi xuống theo cạnh cây khi có thể, nếu không thì theo con trở tiền tố để lùi. Theo phân tích khấu hao, thời gian tỉ lệ với số ký tự đã quét. Để trình bày gọn, từ đây ta xét bài toán tìm mọi vị trí xuất hiện của mọi mẫu.

```
Function ScanB(L, R)
  scan T[L..R] from right to left in TreeB
  stop when no path matches, or when reaching L
  return the last text position scanned
End
```

Nếu hàm này trả về L , đoạn $T[L..R]$ là xâu con của một mẫu nào đó. Ta gọi đoạn như vậy là **hợp lệ**; ngược lại là **không hợp lệ**.

6.4 Quy trình chính

Algorithm MultiPatternMatching2

```

preprocess: build TreeA and TreeB
L <- 0
R <- shortest_pattern_length
point <- TreeA.Root
while R <= n do
  pos <- ScanB(L+1, R)
  if pos > L + 1 then
    point <- TreeA.Root
    (L, point) <- ScanA(pos, R, point)
  R <- L + point.Shift
End

```

Tư tưởng cơ bản là như sau. Điểm bắt đầu đang xét là $L + 1$. Nếu tại đó có một mẫu xuất hiện, đuôi của mẫu phải nằm sau R . Ta quét ngược từ R bằng ScanB và giả sử nó dừng ở pos .

Nếu $pos = L+1$, đoạn $T[L+1..R]$ là hợp lệ, nên có thể tiếp tục quét tiến bằng ScanA từ $L + 1$.

Nếu $pos > L+1$, thì mọi vị trí $L + 1, L + 2, \dots, pos - 1$ đều không thể là đầu của một lần khớp mẫu. Thật vậy, nếu tại một vị trí x trong khoảng đó có mẫu xuất hiện, đoạn $T[x..R]$ phải là hợp lệ, mâu thuẫn với việc ScanB dừng tại pos . Đoạn $T[pos..R]$ là hợp lệ, nên ta đặt lại $point$ về gốc của TreeA rồi quét tiến từ pos .

Điều kiện dừng của ScanA,

$$point.Shift \geq \left\lfloor \frac{\theta}{2} \right\rfloor,$$

với θ là độ dài mẫu ngắn nhất, nhằm bảo đảm ở giai đoạn kế tiếp $R - L$ có cỡ ít nhất $\theta/2$.

6.5 Một ví dụ

Lấy hai mẫu $abcd$ và $bcde$. Cây tiền tố TreeA gồm hai đường:

$$\begin{array}{ccccccc}
 1 & \xrightarrow{a} & 2 & \xrightarrow{b} & 3 & \xrightarrow{c} & 4 \xrightarrow{d} 5, \\
 1 & \xrightarrow{b} & 6 & \xrightarrow{c} & 7 & \xrightarrow{d} & 8 \xrightarrow{e} 9.
 \end{array}$$

Các nút 5 và 9 là nút Okay. Bảng Shift là:

nút	1	2	3	4	5	6	7	8	9
Shift	4	3	2	1	1	3	2	1	4

Giả sử văn bản $T = abcbcbde$.

Giai đoạn 1: $L = 0, R = 4, point = 1$. Quét ngược bằng ScanB từ R về L . Chỉ đoạn a là hợp lệ, còn ca không hợp lệ, nên $pos = 4$. Vì $pos > L+1$, các vị trí 1..3 không thể bắt đầu một mẫu; nếu có, mẫu ấy phải chứa đoạn ca , nhưng ca không là xâu con của mẫu nào. Sau đó ScanA quét đoạn $T[4..4] = a$, đưa $point$ từ 1 đến 2. Vì $Shift[2] = 3$ không nhỏ hơn $\lfloor 4/2 \rfloor$, giai đoạn kết thúc và chưa tìm thấy mẫu.

Giai đoạn 2: $L = 4, R = L + Shift[point] = 7, point = 2$. Quét ngược thấy bcd hợp lệ và dừng tại $pos = 5 = L+1$, nên không cần đặt lại $point$. Quét tiến $T[5..7]$ đưa $point$ đến nút 5 và phát hiện mẫu $abcd$. Vì $Shift[5] = 1 < \lfloor 4/2 \rfloor$, ScanA đọc thêm ký tự e , đi qua các con trở và cạnh tương ứng đến nút 9, rồi phát hiện tiếp mẫu $bcde$. Khi hết văn bản, thuật toán dừng.

6.6 Phân tích độ phức tạp

Xây cây tiền tố từ và cây hậu tố tốn

$$O(26 \sum L),$$

trong đó bộ nhớ vẫn có thể là nút thắt cổ chai như đã nêu ở Chương 4. Ta tập trung phân tích quá trình quét văn bản.

Gọi θ là độ dài mẫu ngắn nhất. Trường hợp xấu nhất, ScanA có thể phải quét toàn bộ văn bản vì Shift luôn nhỏ hơn $\theta/2$. Khi đó thuật toán vẫn tốn $O(n)$ thời gian.

Khi mọi mẫu đều có độ dài θ , θ đủ lớn, văn bản ngẫu nhiên và bảng chữ cái đủ lớn, xác suất để ScanB quét hết $T[L+1..R]$ không lớn. Khi ấy độ phức tạp trung bình nhỏ hơn nhiều so với $O(n)$. Ta tính như sau.

Giả sử mọi mẫu có độ dài θ và

$$M = \sum L = \theta^k,$$

trong đó k là hằng số nhỏ vì θ lớn. Gọi α là kích thước bảng chữ cái, chẳng hạn 26. Với θ và α đủ lớn, giả sử

$$\log_{\alpha} M^3 \leq \frac{\theta}{2}, \quad \text{tức} \quad 3k \log_{\alpha} \theta \leq \frac{\theta}{2}.$$

Một lần ScanB cộng với một lần ScanA được gọi là một giai đoạn.

Mệnh đề 1. Trong mỗi giai đoạn, $R - L$ có bậc $O(\theta)$.

Lần đầu vào vòng lặp, $L = 0$ và $R = \theta$. Các lần sau, vì ScanA chỉ dừng khi $\text{Shift} \geq \theta/2$, ta có $R \geq L + \theta/2$. Do đó $R - L$ luôn có cỡ $O(\theta)$ trong phân tích này.

Mệnh đề 2. Số giai đoạn nhiều nhất là $O(n/\theta)$. Điều này suy ra trực tiếp từ mệnh đề 1.

Mệnh đề 3. Tập mẫu P có không quá θ^{2k} xâu con, tức không quá chừng ấy xâu hợp lệ.

Mỗi mẫu có không quá θ^2 xâu con. Số mẫu là θ^{k-1} , nên tổng số xâu con không vượt quá một cận cùng bậc; trong lập luận của tác giả, dùng cận rộng θ^{2k} là đủ.

Mệnh đề 4. Tồn tại hằng số C sao cho một xâu gồm $C \log_{\alpha} \theta$ ký tự ngẫu nhiên có xác suất không quá $1/M$ để là xâu con của một mẫu trong P .

Theo mệnh đề 3, số xâu hợp lệ không quá θ^{2k} . Số xâu độ dài $C \log_{\alpha} \theta$ là

$$\alpha^{C \log_{\alpha} \theta} = \theta^C.$$

Vì các khả năng được đọc vào có xác suất như nhau, xác suất đọc được xâu hợp lệ không vượt quá

$$\frac{\theta^{2k}}{\theta^C}.$$

Muốn giá trị này không quá $1/M = 1/\theta^k$, chỉ cần $C \geq 3k$. Chọn $C = 3k$ là một hằng số; giả thiết ở trên cũng cho $C \log_{\alpha} \theta \leq \theta/2$.

Mệnh đề 5. Trong ScanA, khi xử lý đến nút p ở độ sâu x trên TreeA và tương ứng với ký tự văn bản $T[y]$, đoạn $T[y-x+1..y]$ là hợp lệ.

Thật vậy, đoạn này chính là xâu tạo bởi đường đi từ gốc TreeA đến p . Nó là tiền tố của một mẫu, do đó là xâu con của một mẫu.

Mệnh đề 6. Trong ScanA, số ký tự kỳ vọng cần đọc thêm ngoài $[L..R]$ cho đến khi gặp nút có Shift ít nhất $\theta/2$ là

$$O(\log_\alpha \theta).$$

Nếu ScanA dừng trong $C \log_\alpha \theta$ ký tự đầu, kết quả hiển nhiên. Nếu nó dừng trong khoảng từ $C \log_\alpha \theta$ đến $2C \log_\alpha \theta$, thì sau $C \log_\alpha \theta$ ký tự đầu, con trỏ trong TreeA phải có độ sâu ít nhất $\theta/2$; nếu độ sâu nhỏ hơn, vì $C \log_\alpha \theta \leq \theta/2$, giá trị Shift đã đủ lớn để dừng. Theo mệnh đề 5, xâu dài $C \log_\alpha \theta$ vừa tạo là hợp lệ, xác suất của sự kiện này không quá $1/M$ theo mệnh đề 4.

Tương tự, xác suất để dừng trong khoảng $x C \log_\alpha \theta$ đến $(x+1) C \log_\alpha \theta$ giảm theo lũy thừa của $1/M$. Vì vậy kỳ vọng được chặn bởi một tổng hình học:

$$\sum_{x \geq 0} (x+1) C \log_\alpha \theta \cdot \frac{1}{M^x} = O(\log_\alpha \theta).$$

Mệnh đề 7. Số ký tự kỳ vọng được đọc trong mỗi giai đoạn là

$$O(\log_\alpha \theta).$$

Chia làm hai trường hợp.

Trường hợp 1: ScanB đọc không quá $C \log_\alpha \theta$ ký tự. Theo mệnh đề 4, xác suất của trường hợp này ít nhất $1 - 1/M$. Khi đó ScanA bắt đầu từ gốc của TreeA và đọc nhiều nhất $C \log_\alpha \theta$ ký tự nữa; vì $C \log_\alpha \theta \leq \theta/2$, tham số Shift chắc chắn đạt $\theta/2$. Tổng số ký tự không quá $2C \log_\alpha \theta$.

Trường hợp 2: ScanB vẫn chưa dừng sau $C \log_\alpha \theta$ ký tự. Xác suất không vượt quá $1/M$. Khi đó kỳ vọng số ký tự đọc vào không vượt quá θ cho ScanB, θ cho phần ScanA bên trong đoạn, cộng thêm $C \log_\alpha \theta$ ký tự đọc thêm do điều kiện Shift.

Do đó số ký tự kỳ vọng trong một giai đoạn là

$$\left(1 - \frac{1}{M}\right) 2C \log_\alpha \theta + \frac{1}{M} (2\theta + C \log_\alpha \theta) = O(\log_\alpha \theta).$$

Kết hợp mệnh đề 2 và mệnh đề 7, độ phức tạp trung bình của thuật toán là

$$O\left(\frac{n \log_\alpha \theta}{\theta}\right).$$

Hằng số ẩn khá lớn, nên ưu thế của thuật toán chỉ thể hiện rõ khi θ và α đủ lớn.

7 Gợi ý và kết luận

7.1 Phân tích thuật toán

Trong thuật toán chính thứ hai, số $\theta/2$ mang hai ý nghĩa cùng lúc:

- nó là ngưỡng để ScanA dừng;
- nó là cận dưới cho độ dài $R - L$ trong mỗi giai đoạn.

Nếu tăng ngưỡng này, ScanA khó dừng hơn. Nếu giảm nó, số giai đoạn sẽ tăng. Cả hai đều làm độ phức tạp trung bình tăng. Vì vậy $\theta/2$ là điểm trung gian vừa đủ để đạt độ phức tạp trung bình nhỏ nhất trong phân tích này.

7.2 Gợi ý phương pháp

Từ thuật toán này có thể rút ra hai gợi ý:

- phối hợp nhiều thuật toán hoặc cấu trúc dữ liệu, như cây tiền tố từ và cây hậu tố trong bài này;
- chọn điểm cân bằng thích hợp giữa hai yếu tố đối nghịch.

Trong bài này ta dùng trung bình cộng để chọn điểm cân bằng. Thường gặp hơn là trung bình nhân. Một ví dụ điển hình là bài *Editor* ở ngày thi thứ nhất NOI 2003: trong một mảng ký tự dài tối đa $2M$, liên tục thực hiện chèn, xóa, định vị, đọc, v.v., và cần mô phỏng quá trình đó.

Nếu dùng mảng, chèn và xóa tốn $O(2M)$, còn định vị tốn $O(1)$. Nếu dùng danh sách liên kết, chèn và xóa tốn $O(1)$, còn định vị tốn $O(2M)$. Cả hai cách đều không đủ mạnh với giới hạn dữ liệu. Ta phối hợp chúng bằng danh sách liên kết theo khối:

$$\boxed{\text{mảng khối 1}} \rightarrow \boxed{\text{mảng khối 2}} \rightarrow \dots \rightarrow \boxed{\text{mảng khối } Y}.$$

Mỗi khối là một mảng. Đặt kích thước khối là X và số khối là Y ; lý tưởng thì XY xấp xỉ độ dài văn bản. Khi chèn hoặc xóa, ta thao tác vụn trong một vài khối với chi phí $O(X)$, rồi chỉnh con trỏ danh sách với chi phí $O(1)$. Khi định vị, ta tìm khối với chi phí $O(Y)$ rồi tìm vị trí trong khối với chi phí $O(1)$.

Chọn $X = Y = \sqrt{2M}$ tạo điểm cân bằng và hạ độ phức tạp tổng thể.

7.3 Kết luận

Việc cài đặt thuật toán chính thứ hai đã vượt xa độ khó thông thường của OI; hơn nữa thời gian đọc mảng ký tự có thể đã lớn hơn thời gian xử lý chính. Khi đã có thuật toán chính thứ nhất đơn giản và dễ dùng, thuật toán thứ hai còn ý nghĩa thực tế không?

Có. Trước hết, các thuật toán chuẩn bị được dùng ở đây đều đã trở thành công cụ chủ lưu trong thi tin học; nắm vững chúng là năng lực cần có. Tiếp theo, độ phức tạp của thuật toán thứ hai phản ánh ở một mức độ khác rằng cán cân mâu thuẫn giữa phần cứng và phần mềm lại nghiêng về phía phần mềm: phương tiện lưu trữ và thuật toán lưu trữ hiệu quả hơn vẫn cần được tiếp tục phát triển.

Điều đáng quý nhất là tư tưởng “dùng tổng hợp, mở từ điểm yếu nhất” trong cách hình thành thuật toán. Độ bền của một chuỗi phụ thuộc vào mắt yếu nhất; lượng nước trong thùng phụ thuộc vào tấm ván ngắn nhất. Chỉ cần liên tục đột phá ở nút thắt, ta sẽ luôn tìm được đường đi qua bài toán.

Tin học là một ngành sâu rộng. Bản chất của nó không nằm ở kỹ thuật lập trình tầng thấp, mà nằm ở đổi mới tư duy của con người; đó cũng là mục đích của các kỳ thi học thuật.

Lời cảm ơn

Tác giả cảm ơn toàn thể các thầy cô trong nhóm huấn luyện của Ủy ban Olympic Tin học thanh thiếu niên tỉnh Giang Tô; cảm ơn các thầy cô của Trường Ngoại ngữ Nam Kinh đã hỗ trợ; cảm ơn bạn Xin Tao ở Liêu Ninh đã giúp đỡ nhiều; cảm ơn gia đình đã quan tâm; và cảm ơn tất cả bạn bè trong cộng đồng tin học.

References

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*.

- [2] Donald E. Knuth, *The Art of Computer Programming*.
- [3] Allen B. Tucker et al., *The Computer Science and Engineering Handbook*.
- [4] E. Ukkonen, *On-line Construction of Suffix Trees*, 1995.
- [5] Barbara König, *McCreight's Suffix Tree Construction Algorithm*.
- [6] Wu Wenhui and Wang Jiande, *Analysis and Programming of Practical Algorithms*.